

Rob The Block: Phase 3 Report

Unit and Integration Tests:

- We wrote unit tests for each file ensuring line and branch coverage
- Created unit test for all main classes:
 - Enemies: Dog, Homeowner and generic enemy
 - Level: Tile Testing
 - Object: Rewards, dog toy, punishments, different types of rewards
- Some tests focused on unit testing such as:
 - Position updating
 - Reward values
- Some tests focused on integration testing such as:
 - Burglar and the punishments and also rewards
 - Tiles and if there are rewards
 - Enemy and player collision
- Some test focused on boundary testing such as:
 - Score boundary tests
 - Grid boundary tests

Test Automation:

All tests were used using JUnit framework

- Having the structure of:
 - @BeforeEach
 - @Test for test cases

Tests were also put into packages for more organization and cleaner code
Assertions were used to test.

Testing Methodology:

- Unit Testing: We focused on unit testing individual methods in isolation. For example, BurglarTest tests the updateScore() method independently from movement, and PositionTest validates coordinate changes without requiring a full game level.
- Integration Testing: Several tests verify component interactions. The move() and moveDiagonal() tests require a Level object to validate collision detection; ensuring the Burglar and Level classes work together correctly. Enemy collision tests verify that Enemy and Burglar position tracking integrates properly.

- Boundary Testing: Tests include edge cases like negative scores (testing `isOutOfCoins()`), movement into walls (testing blocked movement), and zero delta values in `updateSmooth()` (testing rendering edge cases).

Test Quality and Coverage:

Jacoco helped greatly with assessing branch and line coverage. Unit tests were able to get great coverage across packages as described below-

- Characters reached 99% instruction coverage and 84% branch coverage.
 - The remaining 1% uncovered instruction in characters is in the `moveDiagonal()` method, specifically branches where both directional inputs (like UP + LEFT) results in invalid positions when the burglar is against a corner wall. The 16% uncovered branches are split between the `updateSmooth()` method's frame interpolation logic and some diagonal movement paths. The `updateSmooth()` method calculates smooth transitions between tiles based on delta time, and certain branches only execute when the time between frames falls within very specific ranges that are difficult to reproduce consistently in tests.
- Enemies achieved 95% instruction and 78% branch coverage.
 - For enemies, the 5% uncovered instruction is primarily in the Dog class's waiting state logic. When a dog reaches a toy, it enters a waiting state with a counter that decrements each frame. Some branches only trigger when the counter hits exact values during state transitions. The Homeowner class also has uncovered code in `setPatrolPath()` for handling malformed route data. The 22% uncovered branches are in the `moveToward()` method's directional logic, where certain combinations of enemy position and target position create rare movement paths that our tests don't cover.
- Level reached 99% instruction and 90% branch coverage.
 - Lines not included in coverage -
 - individual lines for each grid were not tested as they would be redundant after testing all the possible interactions with the grid. Grid creation was tested for each level.
 - Branches not included in coverage -
 - If else statements - The only if else statement missed was the one which is present to ensure that the grid always remains within bounds. Since the grid is hard-coded it will always remain within bounds so the if statement is just an added layer of security in case we auto generate the grids in the future.
 - Switch statement - All the switch cases have been tested individually but it is not included in the coverage as there is not a default case, and the reason there isn't a default case is because there is no other value possible for the variable that is being used in switch. Objects achieved 97% instruction coverage.

- Objects achieved 97% instruction coverage.
This gap in coverage is attributed to the interface `GameObject` not having some methods. However, all their implementations in the subclasses are tested.

The remaining uncovered areas represent edge cases in frame timing and unusual positioning scenarios rather than typical gameplay. Testing frame-dependent rendering requires mocking libGDX's delta time system, and covering every possible position combination would need hundreds of additional tests for minimal gain. The test suite thoroughly covers the critical functionality (movement validation, collision detection, score tracking, reward collection, and enemy chase behavior) ensuring the game mechanics work correctly in all normal gameplay situations.

Findings:

Intuitive testing is not sufficient to test all the cases. Line coverage alone did not highlight all the issues. Incorporating branch coverage tests highlighted some issues with the code-

- There were issues with movement
- We had methods that were not being tested.

Integration testing also helped us find bugs. An example is that remove reward was removing the reward from the grid but not updating the tile status.